

Capítulo 16

Introdução a Classes e Objetos

Exercícios — Resolução Didática com Código C++

“Neste documento resolvemos os Exercícios 16.5 a 16.15 do Capítulo 16. As cinco primeiras questões (16.5 a 16.10) são conceituais; as cinco últimas (16.11 a 16.15) introduzem nossas primeiras classes completas em C++, cada uma com a estrutura clássica de três arquivos: `<Classe>.h` (declaração), `<Classe>.cpp` (implementação) e `main_<Classe>.cpp` (programa de teste). Todos os códigos foram compilados com `g++ -Wall -Wextra -std=c++14` sem warnings.”

1 Exercício 16.5 — Protótipo versus definição

Enunciado. Explique a diferença entre um protótipo de função e uma definição de função.

Resposta

Um **protótipo** (*function prototype*) é a *declaração* da função: anuncia ao compilador o nome da função, o tipo de retorno e os tipos (e opcionalmente os nomes) de seus parâmetros. Não contém corpo — apenas a assinatura, encerrada por ponto-e-vírgula.

Uma **definição** (*function definition*) é a forma completa da função: inclui o cabeçalho (mesmo do protótipo, mas sem ponto-e-vírgula) e o *corpo* entre chaves — isto é, o código que efetivamente realiza a tarefa.

```
// Prototipo (geralmente em arquivo .h):
double areaCirculo(double radius);

// Definicao (geralmente em arquivo .cpp):
double areaCirculo(double radius) {
    return 3.14159 * radius * radius;
}
```

Por que essa separação?

A separação protótipo/definição serve a três propósitos:

1. **Compilação incremental.** O compilador precisa apenas do protótipo para gerar o código de uma *chamada* à função — sabe o tipo de cada argumento, sabe o tipo do retorno. Pode-se compilar quem *chama* sem ter o código de quem é *chamado*.
2. **Verificação de tipos.** Antes da existência dos protótipos (em C antigo, K&R), o compilador não podia checar se você estava passando um `double` quando a função esperava `int`; o erro só aparecia em tempo de execução. Os protótipos eliminaram essa classe inteira de bugs.
3. **Encapsulamento.** O protótipo (interface) pode ficar público no `.h`; a definição (implementação) fica privada no `.cpp`. Clientes da função só precisam ver o `.h`.

Boa Prática de Programação

Em código bem organizado, escreva todos os protótipos no início do arquivo (ou em um `.h`) e todas as definições depois. Para classes, esse modelo é ainda mais rígido: a declaração da classe (com protótipos das funções-membro) vai no `.h`; as definições vão no `.cpp`.

2 Exercício 16.6 — Construtor default

Enunciado. O que é um construtor default? Como os dados-membro de um objeto são inicializados se a classe tiver apenas um construtor default definido implicitamente?

Resposta

Um **construtor default** é um construtor que pode ser chamado *sem argumentos*. Ele é usado, por exemplo, quando se cria um objeto assim: `Conta minhaConta;` (sem parênteses, sem argumentos).

Há duas formas de obtê-lo:

- **Definido explicitamente pelo programador:**

```
class Conta {
public:
    Conta() { saldo = 0; }    // construtor default explicito
private:
    int saldo;
};
```

- **Gerado implicitamente pelo compilador:** se a classe *não* declara nenhum construtor, o compilador gera automaticamente um construtor default sem parâmetros e sem corpo.

O que acontece com os dados-membro nesse caso?

Se a classe tem apenas o construtor default *implicitamente gerado pelo compilador*, os dados-membro são inicializados como segue:

- **Dados-membro de tipos fundamentais** (como `int`, `double`, ponteiros): *não são inicializados* — contém valor indefinido (lixo de memória).
- **Dados-membro de tipos de classe** (como `string`): são inicializados chamando o construtor default *daquela tipo*. Por exemplo, uma `string` é inicializada como string vazia.

Erro Comum de Programação

Confiar no construtor default implicitamente gerado para inicializar `ints` e `doubles` é fonte de bugs clássica. Quando você escreve `Conta c;`, se `Conta` não tiver construtor próprio que inicialize `saldo`, então `c.saldo` contém lixo. O primeiro `c.credit(100)` pode resultar em valor caótico.

Boa Prática de Programação

Sempre forneça um construtor explícito que inicialize todos os dados-membro com valores conhecidos. Para classes com múltiplos construtores, garanta que *todos* cubram completamente a inicialização. Em C++11 e posteriores, você também pode inicializar dados-membro diretamente na declaração: `int saldo = 0;`

3 Exercício 16.7 — Finalidade de um dado-membro

Enunciado. Explique a finalidade de um dado-membro.

Resposta

Um **dado-membro** é uma variável declarada como parte de uma classe. Sua finalidade é *armazenar o estado* dos objetos dessa classe.

Discussão expandida

Os dados-membro:

1. **Representam atributos do objeto.** São a contraparte computacional dos “substantivos” do domínio: o *saldo* de uma conta, o *nome* de um aluno, a *data de nascimento* de uma pessoa.
2. **Persistem durante toda a vida do objeto.** Diferentemente de variáveis locais (que vivem apenas durante uma chamada de função), os dados-membro existem desde a construção do objeto até sua destruição.
3. **São individuais por objeto.** Cada objeto tem sua própria cópia de cada dado-membro. *conta1.saldo* e *conta2.saldo* são variáveis distintas.
4. **São compartilhados entre as funções-membro.** Todas as funções-membro de um objeto acessam os mesmos dados-membro daquele objeto, o que permite que diferentes operações cooperem coerentemente sobre o estado interno.
5. **São tipicamente *private*.** Bons projetos OO ocultam os dados-membro — o acesso externo se dá via funções-membro públicas (*getters* e *setters*), o que preserva o *encapsulamento*.

Observação de Engenharia de Software

A oposição entre *dados-membro* (estado) e *funções-membro* (comportamento) é o esqueleto do paradigma orientado a objetos. Outras linguagens chamam isso de *atributos* versus *métodos*, ou *campos* versus *operações*. Os três pares de termos são sinônimos em diferentes tradições; o que importa é a idéia subjacente.

4 Exercício 16.8 — Arquivos de cabeçalho e de código-fonte

Enunciado. O que é um arquivo de cabeçalho? O que é um arquivo de código-fonte? Discuta a finalidade de cada um.

Resposta

Arquivo de cabeçalho (*header file*, extensão *.h*): contém *declarações* — protótipos de funções, definições de classes (com seus dados-membro e protótipos das funções-membro), constantes, definições de tipos. **Não** contém código executável.

Arquivo de código-fonte (*source file*, extensão *.cpp*): contém *definições* — corpos das funções, incluindo as funções-membro das classes. É o código que será efetivamente compilado em código-objeto.

Modelo de divisão típico em C++

Arquivo	Contém	Quem o inclui
Conta.h	Declaração da classe <code>Conta</code>	Todos os arquivos que usam <code>Conta</code>
Conta.cpp	Definições das funções-membro	Compilado uma só vez
main.cpp	Função <code>main()</code> , programa cliente	<code>#include "Conta.h"</code>

Por que essa divisão?

1. **Reutilização.** O `.h` pode ser incluído em quantos arquivos cliente quisermos; a definição `.cpp` é compilada apenas uma vez.
2. **Tempo de compilação.** Se você altera o `.cpp` sem mexer no `.h`, apenas o `.cpp` precisa ser recompilado — os clientes ficam intactos.
3. **Encapsulamento.** Para distribuir uma biblioteca, você fornece o `.h` e um arquivo-objeto compilado (`.o` ou `.lib`). O cliente não precisa ver o código-fonte da implementação.

Boa Prática de Programação

Todo arquivo `.h` deve começar com guardas de inclusão para evitar inclusões múltiplas:

```
#ifndef CONTA_H
#define CONTA_H
// ... declaracao da classe ...
#endif
```

Sem isso, se dois arquivos diferentes incluírem `Conta.h` no mesmo arquivo cliente, a definição da classe será duplicada e o compilador acusará erro.

5 Exercício 16.9 — Usar `string` sem `using`

Enunciado. Explique como um programa poderia usar a classe `string` sem inserir uma declaração `using`.

Resposta

A classe `string` reside no *namespace* `std`. Por padrão, para referência, você precisa *qualificá-la* com o nome do *namespace* usando o operador binário de resolução de escopo:

```
#include <iostream>
#include <string>

int main() {
    std::string nome; // qualificacao completa
    std::cout << "Digite seu nome: ";
    std::cin >> nome;
    std::cout << "Ola, " << nome << std::endl;
    return 0;
}
```

As três alternativas

1. **Qualificação completa** (como acima): `std::string`. Verbosa, mas inequívoca; nunca causa problema.
2. **Declaração `using` específica**: `using std::string;` no início do arquivo. A partir daí, `string` sozinho referencia `std::string`.
3. **Diretiva `using namespace`**: `using namespace std;`. Disponibiliza *todos* os nomes de `std` sem qualificação. Conveniente em programas pequenos, mas problemática em programas grandes (pode introduzir colisões de nomes).

Boa Prática de Programação

Em arquivos `.h`, **nunca** use `using namespace std` no escopo global — isso polui o *namespace* de todo arquivo que inclua o `.h`. Use sempre a qualificação completa `std::string` em cabeçalhos. Em arquivos `.cpp`, `using namespace std` é tolerado mas não recomendado em código de produção.

6 Exercício 16.10 — Por que oferecer *set* e *get*?

Enunciado. Explique por que uma classe poderia oferecer uma função *set* e uma função *get* para um dado-membro.

Resposta

A prática *getter/setter* é o caminho canônico para que clientes acessem dados-membro `private` de modo controlado. Vejamos as quatro razões principais:

1. **Encapsulamento.** Os dados-membro permanecem `private`; a representação interna pode ser alterada (mudar tipo, mudar formato, particionar em vários campos) sem afetar os clientes, contanto que a interface (*getter/setter*) permaneça estável.
2. **Validação no *setter*.** O *setter* pode rejeitar valores inválidos. Ex.: `setSalario(-1000)` pode silenciosamente armazenar 0 (como faremos no Exercício 16.14), ou pode lançar exceção, ou pode reportar erro. O cliente nunca consegue introduzir um estado inválido.
3. **Logging e instrumentação.** O *setter* pode registrar cada modificação para auditoria; o *getter* pode contar acessos para fins estatísticos. Tudo isso ficaria impossível se o cliente acessasse o membro diretamente.
4. **Acesso assimétrico.** Às vezes queremos que um membro seja *legível* mas não *escrevível* de fora. Basta oferecer o *getter* e omitir o *setter*. Em alguns casos, queremos o oposto. Os dados-membro públicos não permitem essa assimetria.

Observação de Engenharia de Software

Cuidado com o uso indiscriminado de *getters* e *setters*: oferecer um *getter* e um *setter* para *cada* dado-membro é, na prática, equivalente a deixar todos os membros públicos — só que mais verboso. O bom projeto OO é *minimalista* na interface pública: expõe apenas os *serviços* que a classe precisa oferecer ao mundo externo, e não seu estado interno bruto.

7 Exercício 16.11 — Modificando a classe GradeBook

Enunciado. Modifique a classe `GradeBook` (figuras 16.11 e 16.12) para: (a) incluir um segundo dado-membro `string` representando o nome do instrutor; (b) fornecer *set/get* para esse nome; (c) modificar o construtor para receber ambos os nomes; (d) modificar `displayMessage` para mostrar uma mensagem de boas-vindas, o nome do curso e o nome do instrutor.

Cabeçalho `GradeBook.h`

```

1 // Exercício 16.11 - GradeBook.h
2 // Declaracao da classe GradeBook com nome do curso E nome do instrutor
3 #ifndef GRADEBOOK_H
4 #define GRADEBOOK_H
5
6 #include <string>
7
8 class GradeBook {
9 public:
10     // Construtor que recebe nome do curso e nome do instrutor
11     GradeBook(std::string nomeDoCurso, std::string nomeDoInstrutor);
12
13     // Funcoes para o nome do curso
14     void setCourseName(std::string nome);
15     std::string getCourseName() const;
16
17     // Funcoes para o nome do instrutor
18     void setInstructorName(std::string nome);
19     std::string getInstructorName() const;
20
21     // Exibe mensagem de boas-vindas
22     void displayMessage() const;
23
24 private:
25     std::string courseName;        // nome do curso
26     std::string instructorName;    // nome do instrutor
27 };
28
29 #endif

```

Implementação `GradeBook.cpp`

```

1 // Exercício 16.11 - GradeBook.cpp
2 // Implementacao das funcoes-membro da classe GradeBook
3 #include <iostream>
4 #include "GradeBook.h"
5 using namespace std;
6
7 // Construtor: inicializa os dois dados-membro
8 GradeBook::GradeBook(string nomeDoCurso, string nomeDoInstrutor)
9     : courseName(nomeDoCurso), instructorName(nomeDoInstrutor) {
10 }
11
12 // Set/get para o nome do curso
13 void GradeBook::setCourseName(string nome) {
14     courseName = nome;
15 }
16
17 string GradeBook::getCourseName() const {

```

```

18     return courseName;
19 }
20
21 // Set/get para o nome do instrutor
22 void GradeBook::setInstructorName(string nome) {
23     instructorName = nome;
24 }
25
26 string GradeBook::getInstructorName() const {
27     return instructorName;
28 }
29
30 // Exibe mensagem de boas-vindas, nome do curso e nome do instrutor
31 void GradeBook::displayMessage() const {
32     cout << "Bem-vindo ao livro de notas para\n" << courseName << "!\n"
33         << "Esse curso e apresentado por: " << instructorName << endl;
34 }

```

Programa de teste main_GradeBook.cpp

```

1 // Exercício 16.11 - main_GradeBook.cpp
2 // Programa de teste da classe GradeBook modificada
3 #include <iostream>
4 #include "GradeBook.h"
5 using namespace std;
6
7 int main() {
8     // Cria dois objetos com nome do curso e nome do instrutor
9     GradeBook livro1("CS101 - Introducao a Programacao em C++",
10         "Prof. Paulo Souza");
11     GradeBook livro2("CS201 - Estruturas de Dados",
12         "Profa. Maria Santos");
13
14     // Exibe as informacoes iniciais
15     cout << "==== Livro 1 ==== \n";
16     livro1.displayMessage();
17     cout << "\n==== Livro 2 ==== \n";
18     livro2.displayMessage();
19
20     // Atualiza o instrutor do livro 1
21     cout << "\n>>> Atualizando o instrutor do Livro 1... \n";
22     livro1.setInstructorName("Prof. Roberto Almeida");
23
24     cout << "\n==== Livro 1 (apos atualizacao) ==== \n";
25     livro1.displayMessage();
26
27     // Demonstra as funcoes get
28     cout << "\n>>> Recuperando via get: \n";
29     cout << "Curso do Livro 2: " << livro2.getCourseName() << "\n";
30     cout << "Instr. do Livro 2: " << livro2.getInstructorName() << "\n";
31
32     return 0;
33 }

```

Execução

Saída da execução

```
$ g++ -Wall -Wextra -std=c++14 GradeBook.cpp main_GradeBook.cpp -o test
$ ./test
==== Livro 1 ====
Bem-vindo ao livro de notas para
CS101 - Introducao a Programacao em C++!
Esse curso e apresentado por: Prof. Paulo Souza

==== Livro 2 ====
Bem-vindo ao livro de notas para
CS201 - Estruturas de Dados!
Esse curso e apresentado por: Profa. Maria Santos

>>> Atualizando o instrutor do Livro 1...

==== Livro 1 (apos atualizacao) ====
Bem-vindo ao livro de notas para
CS101 - Introducao a Programacao em C++!
Esse curso e apresentado por: Prof. Roberto Almeida

>>> Recuperando via get:
Curso do Livro 2: CS201 - Estruturas de Dados
Instr. do Livro 2: Profa. Maria Santos
```

Discussão

- Usamos uma **lista de inicialização de membros** no construtor: `: courseName(nomeDoCurso), instructorName(nomeDoInstrutor)`. Essa é a maneira preferida em C++ moderno para inicializar dados-membro — é mais eficiente que atribuir dentro do corpo do construtor (que requer inicialização default seguida de atribuição).
- As funções *getter* foram marcadas como `const`, indicando que não modificam o estado do objeto. É uma promessa que o compilador faz cumprir.
- Note como `displayMessage` acessa diretamente `courseName` e `instructorName` — esses dados são privados, mas estão no escopo de uma função-membro da própria classe, então são visíveis.

8 Exercício 16.12 — Classe Account

Enunciado. Crie a classe `Account` com saldo `int`, construtor que valida saldo inicial (rejeitando negativo), e três funções-membro: `credit` (soma), `debit` (subtrai validando), `getBalance` (retorna o saldo).

Cabeçalho `Account.h`

```

1  // Exercício 16.12 - Account.h
2  // Declaracao da classe Account
3  #ifndef ACCOUNT_H
4  #define ACCOUNT_H
5
6  class Account {
7  public:
8      // Construtor recebe saldo inicial e valida-o
9      Account(int saldoInicial);
10
11     // Credita um valor ao saldo
12     void credit(int valor);
13
14     // Debita um valor do saldo (validando se ha fundos)
15     void debit(int valor);
16
17     // Retorna o saldo atual
18     int getBalance() const;
19
20 private:
21     int balance;    // saldo da conta
22 };
23
24 #endif

```

Implementação `Account.cpp`

```

1  // Exercício 16.12 - Account.cpp
2  #include <iostream>
3  #include "Account.h"
4  using namespace std;
5
6  // Construtor: valida o saldo inicial
7  Account::Account(int saldoInicial) {
8      if (saldoInicial >= 0) {
9          balance = saldoInicial;
10     } else {
11         balance = 0;
12         cout << "Erro: saldo inicial invalido (" << saldoInicial
13             << "). Saldo definido como 0." << endl;
14     }
15 }
16
17 void Account::credit(int valor) {
18     balance += valor;
19 }
20
21 void Account::debit(int valor) {
22     if (valor > balance) {
23         cout << "Valor do debito superior ao saldo da conta" << endl;
24     } else {

```

```

25     balance -= valor;
26 }
27 }
28
29 int Account::getBalance() const {
30     return balance;
31 }

```

Programa de teste main_Account.cpp

```

1  // Exercício 16.12 - main_Account.cpp
2  // Programa de teste da classe Account
3  #include <iostream>
4  #include "Account.h"
5  using namespace std;
6
7  int main() {
8      // Cria duas contas: uma com saldo valido, outra com saldo invalido
9      cout << "Criando conta1 com saldo inicial 100..." << endl;
10     Account conta1(100);
11
12     cout << "\nCriando conta2 com saldo inicial -50 (invalido)..." << endl;
13     Account conta2(-50);
14
15     cout << "\nSaldos iniciais:" << endl;
16     cout << "    conta1.getBalance() = " << conta1.getBalance() << endl;
17     cout << "    conta2.getBalance() = " << conta2.getBalance() << endl;
18
19     // Testa credit
20     cout << "\n>>> conta1.credit(50)" << endl;
21     conta1.credit(50);
22     cout << "Saldo da conta1: " << conta1.getBalance() << endl;
23
24     cout << "\n>>> conta2.credit(200)" << endl;
25     conta2.credit(200);
26     cout << "Saldo da conta2: " << conta2.getBalance() << endl;
27
28     // Testa debit (valor valido)
29     cout << "\n>>> conta1.debit(30)" << endl;
30     conta1.debit(30);
31     cout << "Saldo da conta1: " << conta1.getBalance() << endl;
32
33     // Testa debit (valor excede o saldo)
34     cout << "\n>>> conta1.debit(500) (excede o saldo!)" << endl;
35     conta1.debit(500);
36     cout << "Saldo da conta1: " << conta1.getBalance() << endl;
37
38     return 0;
39 }

```

Execução

Saída da execução

```

Criando conta1 com saldo inicial 100...

Criando conta2 com saldo inicial -50 (invalido)...
Erro: saldo inicial invalido (-50). Saldo definido como 0.

Saldos iniciais:

```

```
conta1.getBalance() = 100
conta2.getBalance() = 0

>>> conta1.credit(50)
Saldo da conta1: 150

>>> conta2.credit(200)
Saldo da conta2: 200

>>> conta1.debit(30)
Saldo da conta1: 120

>>> conta1.debit(500) (excede o saldo!)
Valor do debito superior ao saldo da conta
Saldo da conta1: 120
```

Discussão

- O construtor faz **validação**: se o saldo inicial é negativo, exibe uma mensagem de erro e adota o valor 0. Essa é a primeira aplicação em código do princípio que estabelecemos no Exercício 16.10: o *setter* (aqui no construtor) garante que o objeto nunca entre em estado inválido.
- A função `debit` demonstra validação adicional: se o valor a debitar excede o saldo disponível, a função recusa a operação (mantém o saldo inalterado) e reporta o problema. Sem essa verificação, teríamos saldos negativos — estado inconsistente.

Observação de Engenharia de Software

Em código de produção, valores monetários são *nunca* representados como `int` simples (não há centavos) nem como `double` (impreciso por arredondamento binário). A prática recomendada é armazenar em centavos como `long long`, ou usar uma classe especializada de *decimal* de ponto fixo. O exercício usa `int` por simplicidade didática.

9 Exercício 16.13 — Classe Fatura

Enunciado. Crie a classe `Fatura` com quatro dados-membro: `numeroPeca` (`string`), `descricao` (`string`), `quantidade` (`int`) e `precoPorItem` (`int`). Construtor inicializa todos. *Set/get* para cada. `obterValorFatura()` calcula o total. Quantidade ou preço não-positivos são ajustados para 0.

Cabeçalho `Fatura.h`

```

1 // Exercicio 16.13 - Fatura.h
2 // Declaracao da classe Fatura
3 #ifndef FATURA_H
4 #define FATURA_H
5
6 #include <string>
7
8 class Fatura {
9 public:
10     // Construtor inicializa os quatro dados-membro
11     Fatura(std::string numeroPeca, std::string descricao,
12           int quantidade, int precoPorItem);
13
14     // Set/get para o numero da peca
15     void setNumeroPeca(std::string n);
16     std::string getNumeroPeca() const;
17
18     // Set/get para a descricao
19     void setDescricao(std::string d);
20     std::string getDescricao() const;
21
22     // Set/get para a quantidade (validada)
23     void setQuantidade(int q);
24     int getQuantidade() const;
25
26     // Set/get para o preco por item (validado)
27     void setPrecoPorItem(int p);
28     int getPrecoPorItem() const;
29
30     // Calcula e retorna o valor total da fatura
31     int obterValorFatura() const;
32
33 private:
34     std::string numeroPeca;
35     std::string descricao;
36     int quantidade;
37     int precoPorItem;
38 };
39
40 #endif

```

Implementação `Fatura.cpp`

```

1 // Exercicio 16.13 - Fatura.cpp
2 #include "Fatura.h"
3 using namespace std;
4
5 Fatura::Fatura(string np, string desc, int qtd, int preco)
6     : numeroPeca(np), descricao(desc) {
7     setQuantidade(qtd); // usa o setter para validar

```

```

8     setPrecoPorItem(preco); // usa o setter para validar
9 }
10
11 void Fatura::setNumeroPeca(string n) { numeroPeca = n; }
12 string Fatura::getNumeroPeca() const { return numeroPeca; }
13
14 void Fatura::setDescricao(string d) { descricao = d; }
15 string Fatura::getDescricao() const { return descricao; }
16
17 void Fatura::setQuantidade(int q) {
18     quantidade = (q > 0) ? q : 0; // se nao positiva, define 0
19 }
20 int Fatura::getQuantidade() const { return quantidade; }
21
22 void Fatura::setPrecoPorItem(int p) {
23     precoPorItem = (p > 0) ? p : 0; // se nao positivo, define 0
24 }
25 int Fatura::getPrecoPorItem() const { return precoPorItem; }
26
27 int Fatura::obterValorFatura() const {
28     return quantidade * precoPorItem;
29 }

```

Programa de teste main_Fatura.cpp

```

1 // Exercício 16.13 - main_Fatura.cpp
2 #include <iostream>
3 #include "Fatura.h"
4 using namespace std;
5
6 void imprimirFatura(const Fatura &f) {
7     cout << "   Peca: "           << f.getNumeroPeca()
8          << "   Descr: "         << f.getDescricao() << "\n"
9          << "   Qtd: "           << f.getQuantidade()
10         << "   Preço unit: "     << f.getPrecoPorItem()
11         << "   Total: "          << f.obterValorFatura() << endl;
12 }
13
14 int main() {
15     // Fatura valida
16     cout << "=== Fatura 1 (valores normais) ===" << endl;
17     Fatura f1("P-1001", "Martelo de aco", 3, 25);
18     imprimirFatura(f1);
19
20     // Fatura com quantidade negativa (sera ajustada para 0)
21     cout << "\n=== Fatura 2 (quantidade negativa: deve virar 0) ===" << endl;
22     Fatura f2("P-1002", "Chave de fenda", -5, 12);
23     imprimirFatura(f2);
24
25     // Fatura com preco negativo (sera ajustado para 0)
26     cout << "\n=== Fatura 3 (preco negativo: deve virar 0) ===" << endl;
27     Fatura f3("P-1003", "Furadeira", 2, -30);
28     imprimirFatura(f3);
29
30     // Demonstrando setters
31     cout << "\n>>> Atualizando f1 com novos valores..." << endl;
32     f1.setQuantidade(10);
33     f1.setPrecoPorItem(15);
34     imprimirFatura(f1);
35
36     return 0;
37 }

```

Execução

Saída da execução

```
=== Fatura 1 (valores normais) ===  
Peca: P-1001  Descr: Martelo de aco  
Qtd: 3  Preço unit: 25  Total: 75  
  
=== Fatura 2 (quantidade negativa: deve virar 0) ===  
Peca: P-1002  Descr: Chave de fenda  
Qtd: 0  Preço unit: 12  Total: 0  
  
=== Fatura 3 (preco negativo: deve virar 0) ===  
Peca: P-1003  Descr: Furadeira  
Qtd: 2  Preço unit: 0  Total: 0  
  
>>> Atualizando f1 com novos valores...  
Peca: P-1001  Descr: Martelo de aco  
Qtd: 10  Preço unit: 15  Total: 150
```

Discussão

- Observe um padrão importante: o construtor *chama os setters* (`setQuantidade`, `setPrecoPorItem`) em vez de atribuir diretamente. Isso garante que a validação seja aplicada *uma só vez* no *setter* — princípio DRY (*Don't Repeat Yourself*). Se a regra de validação mudar no futuro, basta alterá-la em um lugar.
- A função `imprimirFatura` é externa à classe e recebe uma referência constante (`const Fatura &f`). Por que? Evita cópia desnecessária e promete não modificar o objeto.

Boa Prática de Programação

Sempre que um construtor precisa validar dados, *delegue* a validação aos *setters* chamando-os a partir do corpo do construtor. É mais limpo, mais conciso e mais fácil de manter do que duplicar a lógica de validação.

10 Exercício 16.14 — Classe Empregado

Enunciado. Crie a classe `Empregado` com primeiro nome (`string`), sobrenome (`string`) e salário mensal (`int`). Construtor inicializa os três. *Set/get* para cada. Salário não-positivo deve virar 0. Crie dois `Empregados`, mostre o salário anual de cada um, dê um aumento de 10% e mostre os salários anuais novamente.

Cabeçalho `Empregado.h`

```

1  // Exercício 16.14 - Empregado.h
2  #ifndef EMPREGADO_H
3  #define EMPREGADO_H
4
5  #include <string>
6
7  class Empregado {
8  public:
9      Empregado(std::string primeiroNome, std::string sobrenome,
10               int salarioMensal);
11
12      void setPrimeiroNome(std::string n);
13      std::string getPrimeiroNome() const;
14
15      void setSobrenome(std::string s);
16      std::string getSobrenome() const;
17
18      // O setter valida: se nao positivo, define 0
19      void setSalarioMensal(int s);
20      int getSalarioMensal() const;
21
22 private:
23     std::string primeiroNome;
24     std::string sobrenome;
25     int salarioMensal;
26 };
27
28 #endif

```

Implementação `Empregado.cpp`

```

1  // Exercício 16.14 - Empregado.cpp
2  #include "Empregado.h"
3  using namespace std;
4
5  Empregado::Empregado(string pn, string sn, int salario)
6      : primeiroNome(pn), sobrenome(sn) {
7      setSalarioMensal(salario);    // usa setter para validar
8  }
9
10 void Empregado::setPrimeiroNome(string n) { primeiroNome = n; }
11 string Empregado::getPrimeiroNome() const { return primeiroNome; }
12
13 void Empregado::setSobrenome(string s) { sobrenome = s; }
14 string Empregado::getSobrenome() const { return sobrenome; }
15
16 void Empregado::setSalarioMensal(int s) {
17     salarioMensal = (s > 0) ? s : 0;
18 }
19 int Empregado::getSalarioMensal() const { return salarioMensal; }

```

Programa de teste main_Empregado.cpp

```

1 // Exercício 16.14 - main_Empregado.cpp
2 #include <iostream>
3 #include "Empregado.h"
4 using namespace std;
5
6 void exibirEmpregado(const Empregado &e) {
7     cout << "    Nome:      " << e.getPrimeiroNome() << " " << e.getSobrenome()
8         << "\n    Salario mensal: " << e.getSalarioMensal()
9         << "\n    Salario anual:  " << e.getSalarioMensal() * 12 << endl;
10 }
11
12 int main() {
13     // Cria dois empregados
14     Empregado emp1("Ana", "Silva", 3500);
15     Empregado emp2("Carlos", "Pereira", 4200);
16
17     cout << "=== Antes do aumento ===" << endl;
18     cout << "\n-- Empregado 1 --" << endl;
19     exibirEmpregado(emp1);
20     cout << "\n-- Empregado 2 --" << endl;
21     exibirEmpregado(emp2);
22
23     // Aplica aumento de 10% a cada empregado
24     // (truncamento inteiro: salario * 110 / 100)
25     emp1.setSalarioMensal(emp1.getSalarioMensal() * 110 / 100);
26     emp2.setSalarioMensal(emp2.getSalarioMensal() * 110 / 100);
27
28     cout << "\n=== Apos aumento de 10% ===" << endl;
29     cout << "\n-- Empregado 1 --" << endl;
30     exibirEmpregado(emp1);
31     cout << "\n-- Empregado 2 --" << endl;
32     exibirEmpregado(emp2);
33
34     // Testa validacao com salario negativo
35     cout << "\n>>> Tentando definir salario negativo em emp1..." << endl;
36     emp1.setSalarioMensal(-1000);
37     cout << "Salario apos tentativa invalida: "
38         << emp1.getSalarioMensal() << endl;
39
40     return 0;
41 }

```

Execução

Saída da execução

```

=== Antes do aumento ===

-- Empregado 1 --
Nome:      Ana Silva
Salario mensal: 3500
Salario anual:  42000

-- Empregado 2 --
Nome:      Carlos Pereira
Salario mensal: 4200
Salario anual:  50400

=== Apos aumento de 10% ===

```



```
-- Empregado 1 --
Nome:      Ana Silva
Salario mensal: 3850
Salario anual: 46200

-- Empregado 2 --
Nome:      Carlos Pereira
Salario mensal: 4620
Salario anual: 55440

>>> Tentando definir salario negativo em empl...
Salario apos tentativa invalida: 0
```

Discussão

- O aumento de 10% é implementado como `salario * 110 / 100`. Por que essa ordem? `salario * 110` primeiro *umenta* o valor (evitando perda de precisão na divisão por aritmética inteira); depois divide por 100. Se fôssemos calcular como `salario * (110 / 100)`, teríamos `salario * 1` (pois $110/100 = 1$ em aritmética inteira). Erro clássico em iniciantes.
- Para Ana: $3500 \times 110 \div 100 = 385000 \div 100 = 3850$. Salário anual: $3850 \times 12 = 46200$. Confere com a saída.
- A demonstração final do *setter* validador mostra o princípio em ação: tentamos definir um salário negativo e o sistema corrige automaticamente para 0.

Erro Comum de Programação

Misturar tipos em expressões aritméticas pode produzir resultados surpreendentes. `3500 * 1.1` retornaria `double` (3850.0); `3500 * 1.1 + 0.5` arredondaria para 3850; usar essas operações em fluxos monetários reais demanda escolhas conscientes sobre arredondamento. Em produção, use bibliotecas de *decimal* de ponto fixo.

11 Exercício 16.15 — Classe Date

Enunciado. Crie a classe `Date` com mês, dia e ano (todos `int`). Construtor inicializa os três. Suponha ano e dia corretos, mas valide o mês: deve estar em 1–12; senão, define como 1. *Set/get* para cada. Função `mostraData` imprime no formato `mes/dia/ano`.

Cabeçalho `Date.h`

```

1  // Exercício 16.15 - Date.h
2  #ifndef DATE_H
3  #define DATE_H
4
5  class Date {
6  public:
7      Date(int mes, int dia, int ano);
8
9      // setters / getters
10     void setMes(int m);
11     int  getMes() const;
12
13     void setDia(int d);
14     int  getDia() const;
15
16     void setAno(int a);
17     int  getAno() const;
18
19     // Exibe a data no formato mes/dia/ano
20     void mostraData() const;
21
22 private:
23     int mes;
24     int dia;
25     int ano;
26 };
27
28 #endif

```

Implementação `Date.cpp`

```

1  // Exercício 16.15 - Date.cpp
2  #include <iostream>
3  #include "Date.h"
4  using namespace std;
5
6  Date::Date(int m, int d, int a)
7      : dia(d), ano(a) {
8      setMes(m);    // valida o mes via setter
9  }
10
11 void Date::setMes(int m) {
12     if (m >= 1 && m <= 12) {
13         mes = m;
14     } else {
15         cout << "Mes invalido (" << m << "). Definido como 1." << endl;
16         mes = 1;
17     }
18 }
19 int Date::getMes() const { return mes; }
20

```

```

21 void Date::setDia(int d) { dia = d; }
22 int Date::getDia() const { return dia; }
23
24 void Date::setAno(int a) { ano = a; }
25 int Date::getAno() const { return ano; }
26
27 void Date::mostraData() const {
28     cout << mes << "/" << dia << "/" << ano << endl;
29 }

```

Programa de teste main_Date.cpp

```

1  // Exercício 16.15 - main_Date.cpp
2  #include <iostream>
3  #include "Date.h"
4  using namespace std;
5
6  int main() {
7      cout << "=== Data 1 (valida) ===" << endl;
8      Date d1(5, 15, 2026);
9      cout << "Data: "; d1.mostraData();
10
11     cout << "\n=== Data 2 (mes invalido = 13) ===" << endl;
12     Date d2(13, 25, 2025);
13     cout << "Data: "; d2.mostraData();
14
15     cout << "\n=== Data 3 (mes invalido = 0) ===" << endl;
16     Date d3(0, 1, 2024);
17     cout << "Data: "; d3.mostraData();
18
19     cout << "\n>>> Atualizando d1 com setMes(7), setDia(4), setAno(2030)..."
20         << endl;
21     d1.setMes(7);
22     d1.setDia(4);
23     d1.setAno(2030);
24     cout << "Data: "; d1.mostraData();
25
26     cout << "\n>>> Tentando setMes(15) em d1 (invalido)..." << endl;
27     d1.setMes(15);
28     cout << "Data: "; d1.mostraData();
29
30     return 0;
31 }

```

Execução

Saída da execução

```

=== Data 1 (valida) ===
Data: 5/15/2026

=== Data 2 (mes invalido = 13) ===
Mes invalido (13). Definido como 1.
Data: 1/25/2025

=== Data 3 (mes invalido = 0) ===
Mes invalido (0). Definido como 1.
Data: 1/1/2024

>>> Atualizando d1 com setMes(7), setDia(4), setAno(2030)...

```

```
Data: 7/4/2030

>>> Tentando setMes(15) em d1 (invalido)...
Mes invalido (15). Definido como 1.
Data: 1/4/2030
```

Discussão

- Note que a validação do mês é idêntica em duas situações: no construtor (através da chamada a `setMes`) e em chamadas diretas de `setMes(15)` pelo cliente. Em ambos os casos, mês 15 vira 1 e a mensagem de erro é impressa. **Uma só lógica de validação**, executada em qualquer caminho de acesso.
- O construtor usa a lista de inicialização para `dia` e `ano` (que não precisam de validação), mas *chama* `setMes(m)` no corpo — a maneira correta de delegar a validação a um *setter*.

Observação de Engenharia de Software

Esta `Date` é propositadamente simples para fins didáticos. Uma classe `Date` de produção deveria validar o dia em função do mês (fevereiro tem 28 ou 29; abril, junho, setembro e novembro têm 30; os demais 31), tratar anos bissextos, calcular dias entre datas, conhecer fusos horários etc. Em C++ moderno, recorre-se ao cabeçalho `<chrono>` da biblioteca padrão para tudo isso.

Boa Prática de Programação

Padrões de formatação de data variam globalmente: `mm/dd/yyyy` (EUA), `dd/mm/yyyy` (Brasil, Europa), `yyyy-mm-dd` (ISO 8601, padrão internacional recomendado). O formato ISO é preferível em logs, arquivos e APIs — ordena alfabeticamente e elimina ambiguidade. O exercício segue a convenção do livro original (EUA, com barras).

Encerramento

Resolvemos onze exercícios do Capítulo 16. Os cinco conceituais (16.5–16.10) consolidaram a terminologia: protótipo versus definição, construtor default, dado-membro, arquivos `.h/.cpp`, *namespace* `std`, *getters/setters*. Os cinco *de programação* (16.11–16.15) introduziram nossas primeiras classes completas, exercitando o padrão de três arquivos (`.h`, `.cpp`, `main`) e o princípio de validação centralizada em *setters*.

O Capítulo 17, seguindo a metodologia incremental dos autores, aprofundará o tratamento de classes: separação mais cuidadosa entre interface e implementação, construtores sobrecarregados, membros constantes, classes amigas, ponteiro `this` e composição — todos os ingredientes que faltam para escrever código OO de qualidade profissional.